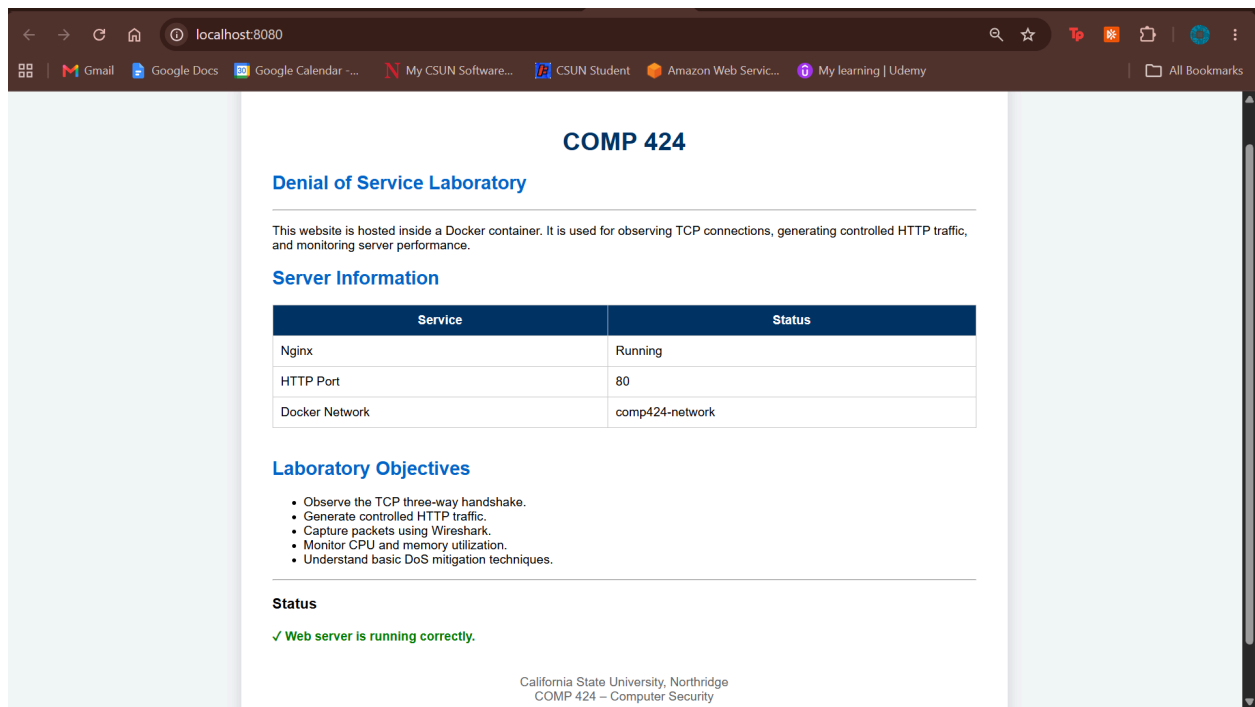




Laboratory Web Page: The screenshot confirms that the COMP 424 Denial of Service laboratory web server was running successfully at `localhost:8080`. The page shows the Nginx service, HTTP port 80, and the private Docker network, verifying that the local containerized lab environment was operating correctly.

```
COMP 424 Client
Analyze the captured traffic and answer the
laboratory questions.

=====
COMP 424 - California State University, Northridge
=====
help          Show laboratory help
light         Light HTTP benchmark
medium        Medium HTTP benchmark
heavy         Heavy HTTP benchmark
pingserver    Ping the web server
curlserver    Download the web page
status        Show server status page
connections   Display TCP connections
myip          Display IP configuration
cls           Clear the screen
exit          Leave the container

=====
student@client:~$ pingserver
PING server (172.18.0.2) 56(84) bytes of data.
64 bytes from comp424-webserver.comp424-network (172.18.0.2): icmp_seq=1 ttl=64 time=19.3 ms
64 bytes from comp424-webserver.comp424-network (172.18.0.2): icmp_seq=2 ttl=64 time=28.5 ms
64 bytes from comp424-webserver.comp424-network (172.18.0.2): icmp_seq=3 ttl=64 time=0.130 ms
64 bytes from comp424-webserver.comp424-network (172.18.0.2): icmp_seq=4 ttl=64 time=0.117 ms

--- server ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3014ms
rtt min/avg/max/mdev = 0.117/12.010/28.457/12.316 ms
student@client:~$
```

```
COMP 424 Client
student@client:~$ curlserver
<!DOCTYPE html>

<html lang="en">
<head>
<meta charset="UTF-8">
<title>COMP 424 DoS Laboratory</title>
<style>
body{
    background:#f4f6f9;
    font-family:Arial,Helvetica,sans-serif;
    margin:40px;
}
.container{
    max-width:900px;
    margin:auto;
    background:white;
    padding:40px;
    border-radius:10px;
    box-shadow:0px 0px 20px rgba(0,0,0,.15);
}
```

Verified Connectivity: The pingserver command successfully reached the web-server container at 172.18.0.2, with four packets sent, four received, and 0% packet loss. The curlserver command also retrieved the webpage's HTML content, confirming that the client could communicate with the server and access its web service over the private Docker network.

Task 1 answers

Was the server reachable?

Yes. The client container successfully reached the server container, as demonstrated by the four successful ping responses and 0% packet loss.

Did the web page load successfully?

Yes. The curlserver command returned the HTML content of the laboratory webpage, and the page also loaded successfully in the browser at <http://localhost:8080>.

- TCP Threeway Handshake

The image shows a Wireshark packet capture titled '*Adapter for loopback traffic capture'. The filter bar at the top is set to 'tcp.port == 8080'. The packet list shows several packets, with packet 4 highlighted in red, indicating it is selected. The packet details pane shows the structure of packet 4: Ethernet II, Internet Protocol Version 6, and Transmission Control Protocol. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
2	1.440753100	:::1	:::1	TCP	76	34616 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
3	1.440893500	:::1	:::1	TCP	76	8080 → 34616 [SYN, ACK] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
4	1.440946300	:::1	:::1	TCP	64	34616 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
5	1.441507400	:::1	:::1	HTTP	837	GET / HTTP/1.1
6	1.441534700	:::1	:::1	TCP	64	8080 → 34616 [ACK] Seq=1 Ack=774 Win=64512 Len=0
7	1.485411300	:::1	:::1	HTTP	244	HTTP/1.1 304 Not Modified
8	1.485472900	:::1	:::1	TCP	64	34616 → 8080 [ACK] Seq=774 Ack=181 Win=65280 Len=0
30	46.488064700	:::1	:::1	TCP	65	[TCP Keep-Alive] 34616 → 8080 [ACK] Seq=773 Ack=181 Win=65280 Len=1
31	46.488111200	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 8080 → 34616 [ACK] Seq=181 Ack=774 Win=64512 Len=0 SLE=773 SRE=774
66	66.545850000	:::1	:::1	TCP	64	8080 → 34616 [FIN, ACK] Seq=181 Ack=774 Win=64512 Len=0
67	66.545906700	:::1	:::1	TCP	64	34616 → 8080 [ACK] Seq=774 Ack=182 Win=65280 Len=0
86	111.548028800	:::1	:::1	TCP	65	[TCP Keep-Alive] 34616 → 8080 [ACK] Seq=773 Ack=182 Win=65280 Len=1
87	111.548078000	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 8080 → 34616 [ACK] Seq=182 Ack=774 Win=64512 Len=0 SLE=773 SRE=774
92	156.563061300	:::1	:::1	TCP	65	[TCP Keep-Alive] 34616 → 8080 [ACK] Seq=773 Ack=182 Win=65280 Len=1
93	156.563087000	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 8080 → 34616 [ACK] Seq=182 Ack=774 Win=64512 Len=0 SLE=773 SRE=774
132	186.548836300	:::1	:::1	TCP	64	8080 → 34616 [RST, ACK] Seq=182 Ack=774 Win=0 Len=0

Frame 4: Packet, 64 bytes on wire (512 bits), 64 bytes captured (512 bits) on interface
Null/Loopback
Internet Protocol Version 6, Src: :::1, Dst: :::1
Transmission Control Protocol, Src Port: 34616, Dst Port: 8080, Seq: 1, Ack: 1, Len: 0

wireshark: Adapter for loopback traffic captureU1RUT3.pcapng Packets: 1420 · Displayed: 16 (1.1%) · Dropped: 0 (0.0%) Profile: Default

TCP Three-Way Handshake: This Wireshark capture shows the TCP connection established between the local client and the Docker web server on port 8080. The client first sent a SYN packet, the server responded with SYN-ACK, and the client completed the connection with an ACK. The :::1 address represents the IPv6 localhost address, confirming that the communication occurred within the local lab environment.

```
COMP 424 Client x + v
</html>student@client:~$ light
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking server (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Finished 500 requests

Server Software:      nginx/1.31.3
Server Hostname:      server
Server Port:          80

Document Path:        /
Document Length:       2015 bytes

Concurrency Level:     10
Time taken for tests:   0.146 seconds
Complete requests:      500
Failed requests:         0
Total transferred:      1124500 bytes
HTML transferred:       1007500 bytes
Requests per second:    3417.12 [#/sec] (mean)
Time per request:       2.926 [ms] (mean)
Time per request:       0.293 [ms] (mean, across all concurrent requests)
```

Light Benchmark Result: The light benchmark sent 500 HTTP requests to the local Docker web server with a concurrency level of 10. All 500 requests completed successfully with zero failed requests. The server processed an average of 3417.12 requests per second, with an average time of 2.926 milliseconds per request. These results show that the server handled the light traffic load successfully and without errors.

```
COMP 424 Client
student@client:~$ medium
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking server (be patient)
Completed 200 requests
Completed 400 requests
Completed 600 requests
Completed 800 requests
Completed 1000 requests
Completed 1200 requests
Completed 1400 requests
Completed 1600 requests
Completed 1800 requests
Completed 2000 requests
Finished 2000 requests

Server Software:      nginx/1.31.3
Server Hostname:      server
Server Port:          80

Document Path:        /
Document Length:       2015 bytes

Concurrency Level:     50
Time taken for tests:  0.401 seconds
Complete requests:     2000
Failed requests:        0
Total transferred:     4498000 bytes
HTML transferred:      4030000 bytes
Requests per second:   4982.49 [#/sec] (mean)
Time per request:      10.035 [ms] (mean)
Time per request:      0.201 [ms] (mean, across all concurrent requests)
Transfer rate:         10942.98 [Kbytes/sec] received
```

Medium Benchmark Result: The medium benchmark sent 2,000 HTTP requests to the local Docker web server with a concurrency level of 50, meaning up to 50 requests were processed simultaneously. All 2,000 requests completed successfully with zero failed requests. The server processed an average of 4,982.49 requests per second, with an average time of 10.035 milliseconds per request. These results show that the server successfully handled the increased traffic load without errors, although the response time increased compared with the light benchmark because more requests were being processed concurrently.

```
COMP 424 Client
student@client:~$ heavy
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking server (be patient)
Completed 500 requests
Completed 1000 requests
Completed 1500 requests
Completed 2000 requests
Completed 2500 requests
Completed 3000 requests
Completed 3500 requests
Completed 4000 requests
Completed 4500 requests
Completed 5000 requests
Finished 5000 requests

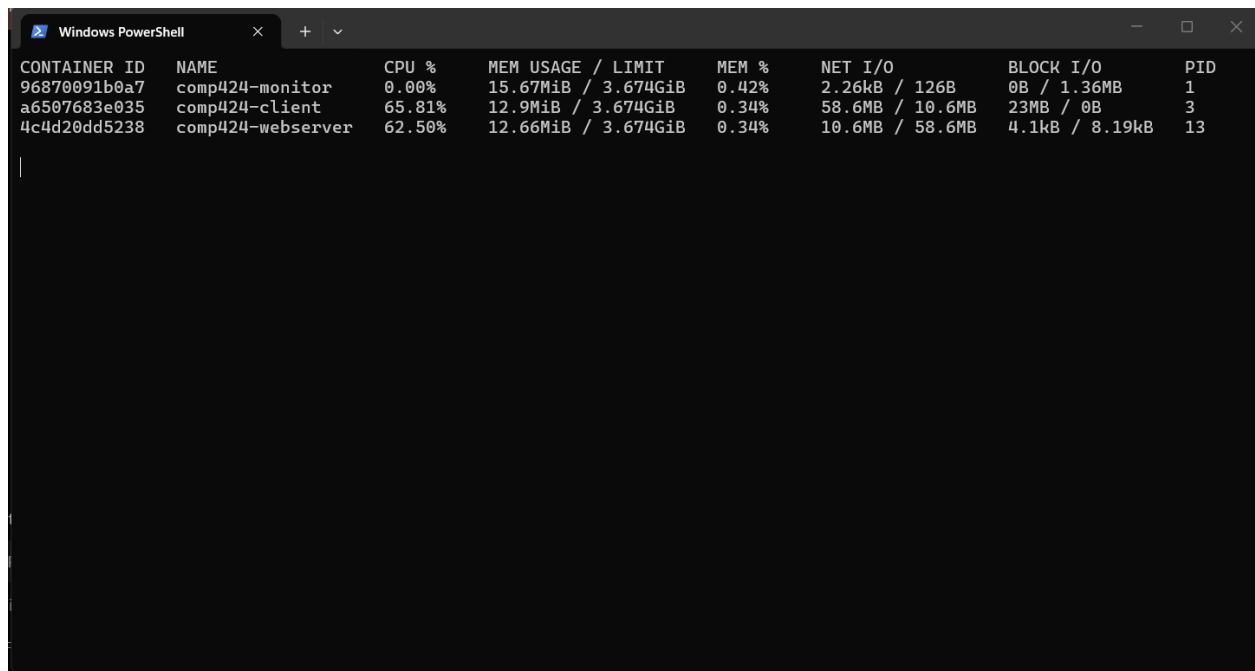
Server Software:      nginx/1.31.3
Server Hostname:      server
Server Port:          80

Document Path:        /
Document Length:       2015 bytes

Concurrency Level:    100
Time taken for tests:  0.904 seconds
Complete requests:    5000
Failed requests:       0
Total transferred:    11245000 bytes
HTML transferred:     10075000 bytes
Requests per second:  5532.46 [#/sec] (mean)
Time per request:     18.075 [ms] (mean)
Time per request:     0.181 [ms] (mean, across all concurrent requests)
Transfer rate:        12150.88 [Kbytes/sec] received
```

Heavy Benchmark Result: The heavy benchmark sent 5,000 HTTP requests to the local Docker web server with a concurrency level of 100, meaning up to 100 requests were processed simultaneously. All 5,000 requests completed successfully with zero failed requests. The server processed an average of 5,532.46 requests per second, with an average time of 18.075 milliseconds per request. Compared with the light and medium tests, the heavy benchmark produced the highest throughput but also the highest individual response time because the server was handling more simultaneous requests. Overall, the server remained available and completed every request without failure.

Test	Requests/sec	Failed Requests
Light	3,417.12	0
Medium	4,982.49	0
Heavy	5,532.46	0

A screenshot of a Windows PowerShell terminal window. The window title is "Windows PowerShell". It displays the output of the 'docker stats' command, showing a table of resource usage for three containers: 'comp424-monitor', 'comp424-client', and 'comp424-webserver'. The table includes columns for Container ID, Name, CPU %, Memory Usage / Limit, Memory %, Network I/O, Block I/O, and PID.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PID
96870091b0a7	comp424-monitor	0.00%	15.67MiB / 3.674GiB	0.42%	2.26kB / 126B	0B / 1.36MB	1
a6507683e035	comp424-client	65.81%	12.9MiB / 3.674GiB	0.34%	58.6MB / 10.6MB	23MB / 0B	3
4c4d20dd5238	comp424-webserver	62.50%	12.66MiB / 3.674GiB	0.34%	10.6MB / 58.6MB	4.1kB / 8.19kB	13

Docker Resource Monitoring: The docker stats command was used to monitor the lab containers while the benchmark was running. The client container reached 65.81% CPU usage while generating HTTP requests, and the web-server container reached 62.50% CPU usage while processing them. The Network I/O values show traffic moving between the client and server, while memory usage remained relatively low. This demonstrates how increased HTTP traffic affects container CPU and network utilization.

Task 5 – Reflection Questions

1. Which benchmark generated the highest server load?

The **heavy benchmark** generated the highest server load because it sent 5,000 requests with a concurrency level of 100. It also produced the highest throughput at **5,532.46 requests per second** and the highest average response time at **18.075 ms**, showing that the server was processing the largest number of simultaneous requests.

2. Which system resource changed the most?

CPU usage changed the most during the benchmark. The comp424-webserver reached **62.50% CPU usage**, while memory usage stayed very low at about **0.34%**, showing that request processing placed much more demand on the processor than on memory.

3. Which protocol carried the benchmark traffic?

The benchmark traffic was carried by **HTTP over TCP**. ApacheBench sent HTTP requests to the local web server, while TCP provided the reliable connections used to deliver the requests and responses.

4. Why does increasing concurrency increase server load?

Increasing concurrency causes more requests to arrive and be processed at the same time. The server must use additional CPU, network capacity, and active connections to handle those simultaneous requests, which increases resource utilization and can also increase response time.